



Literature Review on the Integration of Generative AI in Programming Education

Jemimah Nathaniel^{1,2} · Solomon Sunday Oyelere³ · Jarkko Suhonen¹ · Matti Tedre¹

Received: 23 October 2024 / Accepted: 2 October 2025 / Published online: 24 November 2025
© The Author(s) 2025

Abstract

Integrating generative artificial intelligence tools like ChatGPT and GitHub Copilot into programming education presents notable opportunities to enhance learning but also raises critical challenges in balancing innovation with preserving foundational programming skills. Through a systematic review of 40 empirical studies guided by PRISMA 2020 and Kitchenham’s methodologies, this study evaluates how effectively GenAI was incorporated into programming education and its impact on preserving higher-order thinking skills and foundational programming logic. The Findings reveal that successful integration hinges on intentional teaching strategies, thoughtfully designed assessments, and structured integration processes. However, barriers such as GenAI tools’ limited accessibility features, insufficient bias mitigation, and a tendency to prioritize tool availability over curriculum alignment often disrupt seamless adoption. Additionally, the potential for students to become over-reliant on AI risks diminishing higher-order thinking skills and programming logic. To address these gaps, this paper proposes the GenAI-Ped framework, a structured approach combining self-regulated learning, universal design principles, and iterative feedback to harmonize GenAI-driven support with skill development. The findings emphasized the necessity of strategic implementation, educator training, and inclusive practices to maximize GenAI’s potential in programming education.

Keywords Generative AI · Programming education · Higher-order thinking skills · Programming logic

Introduction

Traditional programming education, primarily delivered in physical classrooms, encounters challenges engaging diverse learners (Sunday et al., 2023), as educators struggle to personalize learning in conventional settings despite available resources, making programming education overwhelming for instructors and students (Oyelere et al., 2017). Overcoming these engagement and personalization challenges is crucial because programming education is also a key pathway for developing students' higher-order thinking skills (HOTS) (Li et al., 2023). Researchers have categorized problem-solving, critical thinking, and creativity as integral components of HOTS (Wu et al., 2024).

According to The National Standards of Higher Education, every university must equip students with the 21st-century skills, including critical thinking, creative thinking, problem solving, and cooperative skills (Setiawan et al., 2018). When students master HOTS, they learn to analyze, organize, express, and evaluate their ideas during complex programming tasks, promoting active participation (Buitrago et al., 2017). Collaborative learning further boosts these skills, as working together has been shown to raise critical-thinking gains among white first-year college students, and also increase motivation and achievement (Loes & Pascarella, 2017; Daungcharone et al., 2023). Although educators have attempted to use appropriate instructional tools or strategies in programming courses to improve students' learning engagement, discrepancies still exist for most educators, as developing students' higher-order thinking skills in programming education is still a considerable challenge (Popat & Starke, 2019; Li et al., 2023).

Over the years, traditional classrooms have embraced various technologies, tools, and innovations to enrich teaching and learning. According to Bomah (2014), technology in the classroom increases student interaction in learning settings for African schools, and integrating technology into the classroom is a common method to improve learners' higher-order thinking (Di et al., 2019). Among the latest technologies, generative AI (GenAI) stands out for its ability to transform several sectors, such as the traditional teaching and learning methods in programming education (Bermejo et al., 2024). GenAI have shown to have the potential to enhance programming through code auto-completion (Phung et al., 2023) and generation (Savelka et al., 2023a), serving as programming assistance (Savelka et al., 2023b), and encouraging personalized learning, and fostering greater student engagement.

However, the integration of GenAI still presents several issues in the literature that need to be addressed. These issues include ensuring the accuracy and reliability of generated code, mainly when catering to learners with varying skill levels (Phung et al., 2023). GenAI also faces the challenge of ensuring that educational tools are universally accessible, considering the potential technological barriers and the digital divide that exists among learners (Yilmaz & Yilmaz, 2023a; Zhang et al., 2022). Another key challenge is assessing the impact of utilizing GenAI tools on long-term retention, mastery, and application of programming skills (Božić, 2023; Sarsa et al., 2022), to improve the understanding of programming logic. Since it can sometime be difficult for the GPT models to replicate higher-order thinking skills, such as critical thinking, problem-solving, and creativity (Li, et al., 2023), it is essential to balance

this innovation in a manner that does not compromise the understanding of programming logic and higher-order thinking skills.

To address this gap, this research aim to comprehensively analyze and synthesize existing literature, exploring successes, challenges, and gaps in integrating GenAI, specifically within programming education. This study applies Kitchenham and Brereton (2013) and Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 guidelines for the filtering process to answer the following research question: How seamless was the integration of GenAI tools into programming education without compromising foundational programming logic and higher-order thinking skills? This study offers important insight for educators and policymakers in enhancing the quality of programming education, thus ensuring that the students develop essential programming skills.

Theoretical Background

Over the past few decades, programming education has evolved significantly, reflecting rapid technological advancements and shifts in educational paradigms. This literature review critically examines various methods and tools employed in teaching programming, the skills cultivated through programming education, and the impact of emerging technologies such as GenAI on programming education.

Generative AI in Programming Education

GenAI is poised to redefine education by enabling open-ended, context-aware interactions that transcend the limitations of earlier educational technologies. This evolution began with foundational systems like Papert's Logo. Papert's Logo system, grounded in constructivist theory, enabled children to explore mathematical and computational ideas through turtle graphics, fostering "learning by doing" in a highly interactive environment (Solomon & Papert, 1976; Solomon et al., 2020). In the 1980s and 1990s, Intelligent Tutoring Systems (ITS) such as the Karel the Robot tutor (Nwana, 1990) and SQL-Tutor provided step-by-step guidance, diagnosing student misconceptions and offering adaptive hints in language-specific contexts (McDougall et al., 2014). Constraint-based tutors (ACT Programming Tutor) and bug-diagnosis tools (Whyline) further helped novices overcome syntactic and semantic errors by detecting common mistake patterns. Throughout the 1990s and 2000s, a rich ecosystem of Intelligent Tutoring Systems (ITS) emerged, such as the Karel and SQL Tutor tutors offering step-by-step guidance, diagnosis of common misconceptions, and adaptive feedback tailored to specific programming languages (Chaudhry & Kazim, 2022). While these systems achieved substantial gains in scaffolded learning, they often required extensive domain-specific rule sets and were limited to predefined problem types.

Today, GenAI is a technology capable of synthesizing decades of pedagogical insight into dynamic, personalized, and boundary-pushing educational experiences. Unlike traditional ITS, which depend on handcrafted knowledge bases, GenAI models leverage broad pretraining to provide on-demand tutoring across languages and

problem domains. According to Chen, Huang, Chen, Tseng, and Li (2023), GenAI plays a fundamental role in potentially changing education, showing its adaptable impact on such domains as personalized learning, immediate feedback, assistance, and enhanced motivation (Agbo, et al., 2019a, 2019b). It has also improved software developers' productivity and speed up the prototyping of ideas, liberating time from laborious low-level tasks to higher-level tasks and experimental design (Sallam, 2023).

The integration of GenAI tools has also been applied within the field of programming education, leading to the creation of model tools like Microsoft Copilot, Codex (Finnie-Ansley et al., 2022), Bard (now known as Gemini) (Motlagh et al., 2023), GitHub Copilot (Lau & Guo, 2023), and ChatGPT (Anagnostopoulos, 2023; Motlagh et al., 2023; Phung et al., 2023). Although GenAI has aided code management and production (Sallam, 2023) and has been used for programming tutoring (Zambrano et al., 2023), significant challenges remain in effectively embedding these tools into programming curricula. Challenges still persist in effectively incorporating GenAI into programming education, mainly when catering to learners with diverse learning abilities, and also how to effectively integrate GenAI into programming education without compromising higher-order thinking skills such as problem-solving abilities, critical thinking, and understanding of programming logic (Ragonis & Shmallo, 2022), and the assessment of the impact of utilizing Higher-order thinking skills and GenAI tools on the application of programming skills (Abosalem, 2016; Sarsa et al., 2022; Božić, 2023) and emphasizing the concerns regarding overreliance and cheating (Liu et al., 2024).

To understand the impact of using GenAI to solve programming problems, the study by Singla (2023) and Yilmaz and Yilmaz (2023a) emphasizes the need for a comprehensive understanding of programming logic for proficient application of GenAI tools in programming education. Reiche and Leidner (2023) support this by highlighting students' need to grasp the programming concepts, as students did not possess the programming skills for comprehending and adjusting created code, which is the essential requirement for a strong foundation in programming education (Anagnostopoulos, 2023). These challenges accentuate the importance of providing effective oversight and feedback, particularly through the involvement of experienced teaching assistants.

Teaching Methods in Programming Education

Effective programming education depends on various teaching methods personalized to different learning styles and educational contexts. Before 1950, the teaching of programming meant understanding how computers worked; over time, teaching programming has continued to evolve, generating considerable attention because of rapid technological developments and because it claims to foster other skills such as problem-solving, logical thinking, and creativity (Scherer et al., 2020). The evolution in the programming education field has brought the rise of several research studies conducted to identify the best approach to teaching programming, leading to the development of several methods of teaching programming, as shown in Table 1.

Table 1 Methods, tools, and approaches of teaching programming

Methods	Teaching approach	References
Problem-Based Learning	Engages students by presenting real-world challenges that require active problem solving	Konijn and Hoorn (2020)
Puzzle-Based Learning	Uses puzzles to foster critical thinking and develop problem-solving skills	Merrick (2010), Oyelere et al., (2019)
Pair and Collaborative Programming	Involves students working in pairs or small groups to write code and learn through peer interaction	Zacharias (2010)
Game-Themed (Game-Based) Learning	Integrates interactive, game-like activities to teach abstract programming concepts in a playful context	Sun et al. (2022)
Lectures and Labs	Combines instructor-led lectures with hands-on lab sessions for demonstration and practice	Santos, et al. (2020)
Software Visualization	Uses visual programming languages and tools to represent code and process flows, making abstract concepts more accessible	Kanika et al., (2020)
Use of Robots	Involves programming of robots to perform simple, practical tasks, thus applying programming in tangible, real-world scenarios	Yew and Goh (2016)

Skills Obtained from Programming Education

Teaching programming to students can be challenging because it requires instructors to ensure students understand the foundational programming logic (Ibarra-Torres et al., 2024). Foundational programming logic includes understanding algorithms, data structures, syntax, and the logical flow of code (Jonsson & Tholander, 2022). It helps students build a strong understanding of programming logic, prepares them for advanced topics, and future professional challenges.

The development of higher-order thinking skills is important in programming education, as it advances critical thinking (Smolansky et al., 2023), problem-solving (Kazemitabaar et al., 2023a, 2023b), and creativity (Kothari, 2023). Problem-solving skills equip individuals with the capacity to think computationally and approach challenges from a computational viewpoint, which helps develop learners' critical thinking ability. It involves breaking down complicated problems into manageable steps and developing algorithmic procedures to solve them (Yilmaz & Yilmaz, 2023b). The ability to analyze data and make intelligent decisions is known as critical thinking (Tanty et al., 2022). Palts and Pedaste (2020) highlight that creativity and problem-solving play an essential part in computer programming because they help learners foster and develop the ability to organize and require them to identify and solve problems. These skills eventually forms the basis for advancing one's programming ability and lays the foundation for long-term success in programming education.

Methods

This research aims to comprehensively analyze and synthesize existing literature, exploring successes, challenges, and gaps in integrating GenAI, specifically within programming education. The study searched reputable databases such as Scopus,

ACM, IEEE, and Web of Science. These databases are relevant and contain a vast number of peer-reviewed articles that are relevant to this study. Keywords related to GenAI and programming education and strict criteria to guide the selection of peer-reviewed articles, conference papers, and scholarly publications were used to ensure appropriate results. The Kitchenham guidelines (Kitchenham & Brereton, 2013) and the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 guidelines ensure the highest level of transparency, rigor, and reproducibility. The study developed a comprehensive search strategy that meticulously documented each database, detailing the complete search strings, date ranges, and any filters applied, as illustrated in Figs. 1 and 2, respectively. These methods are widely recognized in academic research and have been used by other researchers (Santos et al., 2020).

Stage 1: Initial Search Phase

This stage investigates whether there are sufficient papers for the study and the selection of papers based on the abstract and title.

Identify the Purpose

Due to the topic's novelty, there were limited systematic literature reviews that examine empirical studies of the research gap in integrating GenAI into programming education. This highlights the importance of addressing this research gap to enhance understanding of how the integration of GenAI has been utilized in programming education.

Specifying the Research Questions

This study used the inclusion criteria for the specific year range from 2010 to February 2024, narrowing down to articles that address the research question on GenAI and programming education. Selected articles were analyzed to highlight current research trends in the emerging field of GenAI and programming education, as summarized

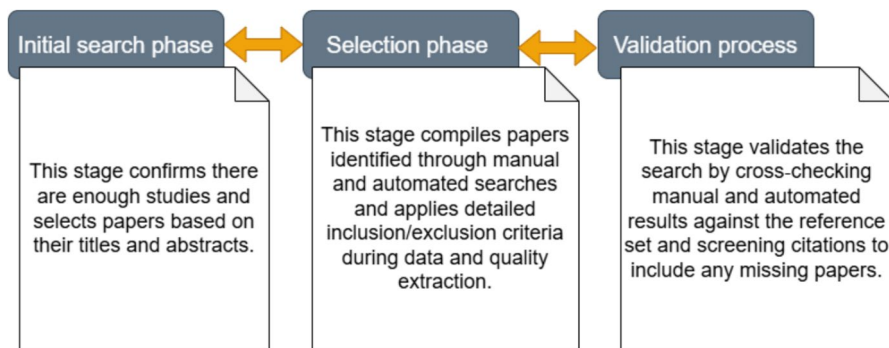


Fig. 1 Systematic literature review process

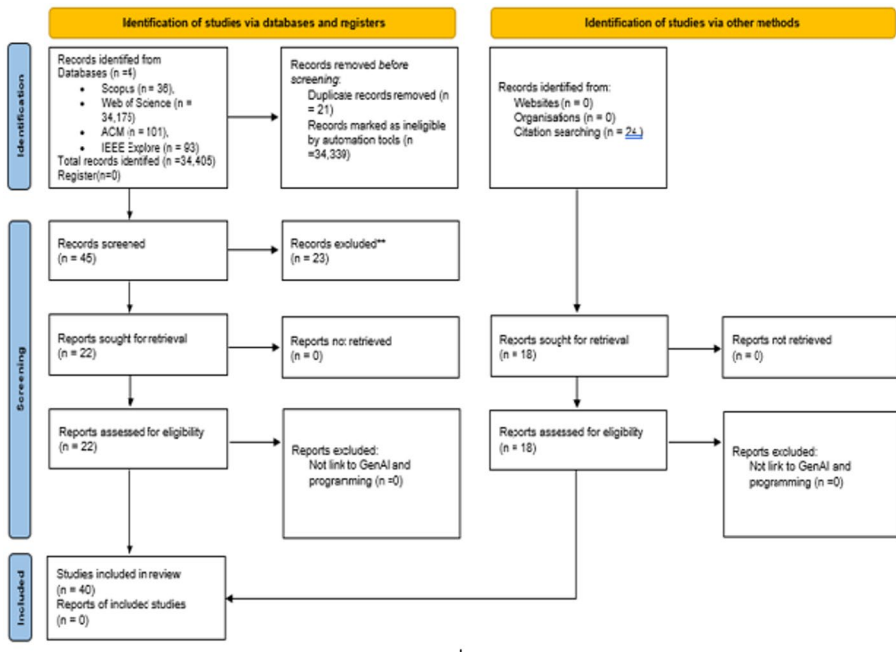


Fig. 2 Overview of processes involved in articles selection

in Table 4. The study framed the research question to guide categorize the papers for data extraction and analysis. Research question (RQ): How seamless was the integration of GenAI tools into programming education without compromising foundational programming logic and higher-order thinking skills?

Protocol for Review

During the initial phase of this review, the study undertook a comprehensive approach by conducting both informal and formal searches to gather relevant information and fulfill the study's research objectives. This method helped to ensure that the study process is transparent and rigorous to reduce any potential bias in data extraction and selection of studies (Xiao & Watson, 2019).

Stage 2: Selection Process

This stage included the list of papers found from known documents and papers agreed for inclusion by a manual and automated search. Table 2 detailed inclusion/exclusion criterion was also carried out during the data extraction and quality extraction processes.

Table 2 The specific protocol executed in each database protocol executed

Database	Database protocol executed
Scopus	("generative artificial intelligence" OR "generative AI" OR "GenAIs") AND ("programming education" OR "coding education" OR "software development education")
IEEE Explore	("generative artificial intelligence" OR "generative AI" OR "GenAIs") AND ("programming education" OR "coding education" OR "software development education")
ACM	("generative artificial intelligence" OR "generative AI" OR "GenAIs") AND ("programming education" OR "coding education" OR "software development education")
Web of Science	("generative artificial intelligence" OR "generative AI" OR "GenAIs") AND ("programming education" OR "coding education" OR "software development education")

Search String

The search strings used in this study were carefully formulated and employed based on the research question. To ensure that all the relevant studies related to our research question were captured, the relevant search strings used in the database search were;

("generative artificial intelligence" OR "generative AI" OR "GenAIs") AND ("programming" OR "programming education" OR "coding education" OR "software development education").

The terms "generative artificial intelligence" OR "generative AI" OR "GenAIs" was used to restrict the papers to just generative AI-related papers. And "programming" OR "programming education" OR "coding education" OR "software development education" restricted the findings to programming education to obtain the maximum coverage of relevant studies.

Table 2 lists the search structure for each database independently, along with the particular procedure used. The Boolean operators "OR" and "AND," which linked enclosed search phrases, were employed to incorporate synonyms and alternative spellings.

Classifying the Papers

The inclusion and exclusion criteria helped to filter the articles accurately; selecting the empirical studies that have incorporated GenAI into programming education contexts. This guides in answering the research question and achieving the objective of this study. Table 3 summarizes the inclusion and exclusion criteria used for the study selection.

Stage 3: Validation Process

This phase Validate the search process by comparing papers found by the manual and automated searches with those in the set of papers. In this stage, references are

Table 3 Eligibility criteria

Criterion	Inclusion	Exclusion
Article Topic	Studies focusing on integrating generative AI in programming education	Studies addressing AI use outside programming education
Article Type	Empirical studies that include practical integration of GenAI	Non-empirical, conceptual, or purely theoretical reports
Publication Status	Peer-reviewed and published research	Unpublished or pre-printed manuscripts
Language	Published in English	Studies not published in English
Publication Year	2010 and February 2024	Studies published before 2010

checked from selected papers, and missed papers are selected as outlined by Kitchenham and Brereton (2013).

Data Extraction and Analysis

This study's data sources were analyzed based on the literature review process for conducting a comprehensive and rigorous review of the available literature on a particular topic (Xiao & Watson, 2019). The literature review process that provides a structured approach to conducting the review, from validating the review protocol to reporting the findings. This process ensures that all relevant literature is identified, assessed for quality, analyzed, and synthesized systematically and transparently, leading to more credible and reliable conclusions.

Synthesize Studies

The study process involved a systematic approach that adheres to the established principles of conducting scientific research (Kitchenham & Brereton, 2013; Moher, 2010; Page et al., 2021). The synthesis stage helped transition the research process from an authorial perspective to a thematic perspective, using the findings from multiple studies to draw meaningful conclusions from the collected database.

Literature Search

In the initial random search keyword, GenAI programming education resulted in 34,405 articles found from the selected databases (36 from Scopus, 34,175 from Web of Science, 101 from ACM, and 93 from IEEE Explore). By refining the search criteria, the number of relevant papers was reduced to 66 across the selected database (9 from Scopus, 5 from Web of Science, 50 from ACM, and 2 from IEEE Explore). The selected articles were screened based on the title and abstract, resulting in the elimination of 23 papers. Afterward, 21 duplicate articles were found, resulting in the final count of 22 relevant publications for our systematic literature review. The study then proceeded with the validation process outlined by Kitchenham and Brereton (2013), which involved checking the references of the selected papers and identifying any missed papers; 18 more relevant papers were found, bringing the total number of eligible papers to 40 papers (Scopus 4, Web of Science 2; ACM 33, and IEEE Explore 1). Figure 2 shows the visual representation of the filtering process, which

complies with PRISMA guidelines 2020 (Page et al., 2021) and Kitchenham and Brereton (2013).

Assessing the Paper Quality

After eligible papers identification, each of these papers do through quality assessment checklist for each paper, as shown in Table 4. This protocol outlines the study's process rigorously and transparently, mitigating researcher bias in study selection and data extraction.

Results

The selected articles were categorized based on various factors, including publication years, implementation details, the GenAI models utilized, the incorporation of GenAI into programming education, the teaching and assessment method used, the integration process employed, and the skills gained. The database successfully retrieved 40 papers (Scopus 4, Web of Science 2, ACM 33, and IEEE Explore 1) found eligible using the specified search string, effectively covering aspects of GenAI and programming education.

The Number of Reviewed Studies and Publication Years

After the inclusion and exclusion criteria, the 40 eligible papers were reviewed to assess how seamless the integration of GenAI was in programming education. The study range considered for inclusion was from 2010 to February 2024, and the results showed a gradual increase in the number of papers published on this topic over the years. Of particular interest was the increase in papers on integrating GenAI into programming education from 2019 onward. This drastic increase shows the growing importance of understanding how to integrate GenAI to ensure its effectiveness in programming education (Fig. 3).

RQ: How Seamlessly was the Integration of Generative AI Tools into Programming Education Without Compromising Foundational Programming Learning Principles and Higher-Order Skills?

The findings of this study explore how seamless the integration of GenAI tools into programming education was without compromising foundational programming logic and higher-order thinking skills (HOTS). The results indicate that skills gained is influenced by numerous factors. These factors include the choice of the GenAI tool,

Table 4 Quality assessment checklist

Assessment criterion	Response options
Does the study answer the research question?	Yes/No/Partially
Does the study describe a method for integrating GenAI into programming education?	Yes/No/Partially
Is the presentation of results comprehensive?	Yes/No/Partially

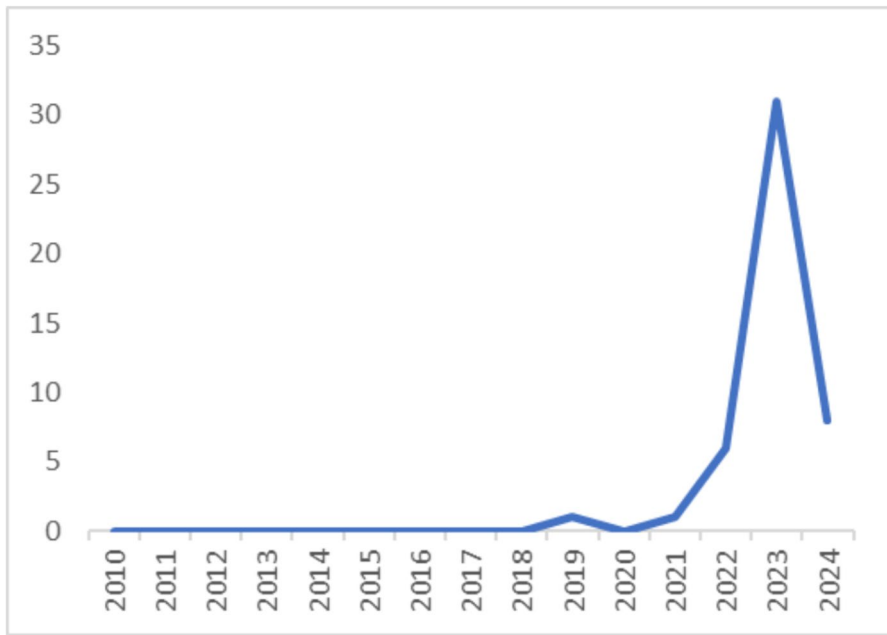


Fig. 3 Overview of the publication year (2010–february 2024)

its alignment with learning objectives, and the effectiveness of the teaching methods used for assessment and integration. By combining all these factors, educators can accurately and comprehensively measure student progress, providing essential feedback for refining the integration process. The teaching methods capitalize on the GenAI tool's capabilities to enhance or maintain foundational programming logic and higher-order thinking skills. This study examined how GenAI was seamlessly integrated into programming education by examining the GenAI tools, teaching, assessment, integration, and skills gained from the selected papers.

Generative AI Tools in Programming Education

Figure 4 shows that ChatGPT is the most commonly used GenAI tool in programming research, likely because of its widespread popularity, easy accessibility, and free availability. ChatGPT has been particularly effective in handling coding-related queries and generating natural language explanations (Xie et al., 2023). Codex and GPT-3.5-turbo, under OpenAI, have also been used in code generation and programming task enhancement (Lewis, 2022). GitHub Copilot, powered by Codex, gets embedded in editors to provide contextual code suggestions (Prather et al., 2023). GPT-4 enhances the previous versions to work better on sophisticated tasks (Doughty et al., 2024). Bidirectional Encoder Representations from Transformers (BERT), developed by Google, outperforms current models and demonstrates potential for human-AI hybrid for categorizing programming problems (Pereira et al., 2021). Their prevalence indicates a trend toward exploiting leading-edge GenAI technologies to boost

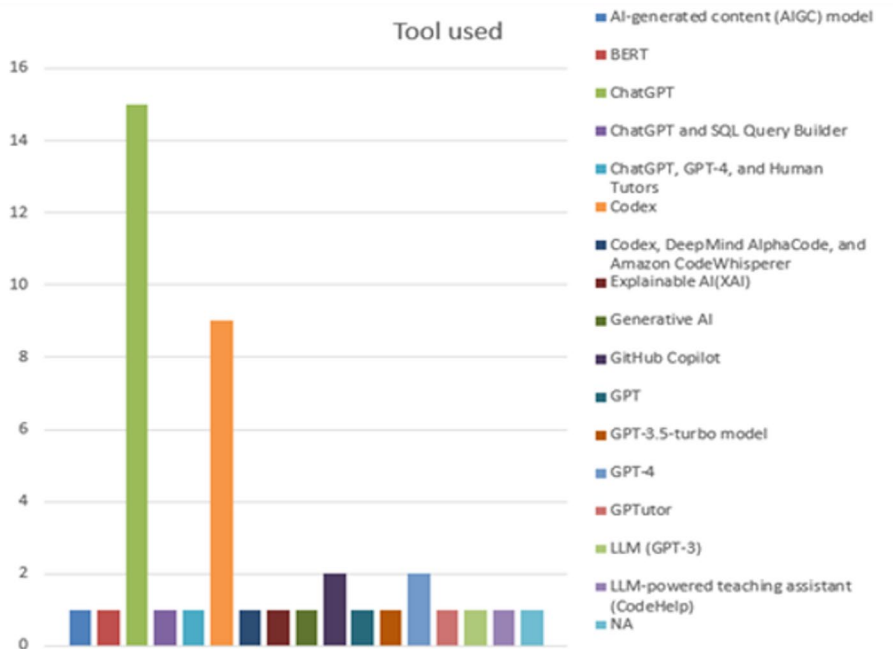


Fig. 4 GenAI tool used

productivity and innovation in programming education. More importantly, their diffusion might be driven by popularity and access rather than by matching specific programming curriculum objectives.

Teaching Methods in GenAI-Enhanced Programming Education

In the growing body of research on integrating GenAI into programming education, several studies focused on how GenAI could enhance or support programming education. For instance, Table 5 illustrates the teaching methods researchers have applied in GenAI-enhanced programming education, demonstrating how GenAI-assisted learning supports active approaches such as workshops and experimental methods (Jonsson & Tholander, 2022), scenario-based design and question-driven learning (Sun et al., 2022), and lectures and exercises (Yilmaz & Yilmaz, 2023a). Kazemitabaar et al., (2023a) and Kazemitabaar et al., (2023b) incorporated embedded programming and AI-assisted learning environments where GenAI provides real-time suggestions, syntax correction, and code efficiency improvement proposals. Additionally, hint-generation modules (Roest et al., 2024), automated assistants (Sheese et al., 2024), and GPTutor code explanations (Chen et al., 2023) have been developed to facilitate students with personalized hints and explanations for self-paced learning, reducing the need for constant interference from the instructor.

Furthermore, to support curriculum preparation and assessment, Finnie-Ansley et al. (2022) and Phung et al. (2023) integrated the use of GenAI into developing auto-

Table 5 Teaching methods and their applications in GenAI-enhanced programming education

Authors	Teaching methods	Application in real-life settings	Pedagogical impact	Significance rating
Ouh et al., 2023; Popovici, 2024; Yilmaz & Yilmaz, 2023a	Code exercises, coding demos, lectures, hands-on lab assignments	GenAI tools assist in real-time code generation and debugging, offering students immediate feedback during exercises and demos	Actively engages students and improves coding fluency with immediate, interactive feedback	Highly Impactful
Kazemitabaar et al., 2023a; Kazemitabaar et al., 2023b	Embedded programming environment, AI-Assisted Learning Environment	GenAI is embedded into coding environments to provide real-time suggestions, assist with syntax correction, and improve code efficiency	Enhances practical skills but should be combined with traditional instruction for a balanced approach	Moderately Impactful
Jonsson and Tholander, 2022; Sun et al., 2022; Wermelinger, 2023	Workshop, scenario-based design, experimental, question-driven Explainable AI	GenAI tools simulate real-world scenarios, facilitating hands-on practice through AI-driven feedback loops	Encourages critical thinking and problem-solving when challenges are well designed	Impactful
Chen et al., 2023; Roest et al., 2024; Sheese et al., 2024	Hint-generation module, automated assistant, GPTutor code explanations	These tools support self-paced learning by generating hints and personalized explanations based on student inputs	Supports self-paced learning while encouraging students to verify and reflect on AI-driven hints	Moderately to Highly Impactful
Berrezueta-Guzman, & Krusche, 2023; Savelka 2023b; Wu et al., 2023	Curriculum preparation, summative assessments, programming education scenarios	GenAI automates feedback and provides AI-driven assessments that align closely with learning outcomes	Strengthens assessment practices and curriculum integration effectively	Highly Impactful
Pereira et al., 2021; Prather et al., 2023; Reeves et al., 2023	Programming exercises, assignments, assessments	GenAI evaluates assignments, offering real-time grading, auto-suggestions for code improvement, and adaptive assessments	Directly enhances practical coding skills and formative assessment through immediate feedback	Impactful

mented feedback to measure the progress of students in programming. These GenAI-powered tools were used to gauge students' learning levels so teachers could create curricula and assessments that matched learning objectives and student performance. For instance, for experimental learning (Pereira et al., 2021; Prather et al., 2023; Reeves et al., 2023), conducted field-tested experiments using the GenAI tool to solve Parsons problems and other programming challenges. Where GenAI automatically suggested code correction that was able to improved problem-solving, making learning programming more effective and practical for real classroom settings. Hence, applying suitable teaching methods when integrating GenAI into programming education provide a structured framework, allowing educators to assess the technical capabilities of GenAI tools and their effectiveness in real-world educational settings.

Assessment Methods in GenAI-Enhanced Programming Education

Assessment methods vary significantly across various studies on integrating GenAI into programming education. This variation arises because several of our selected papers primary objective was to understand the efficacy of the GenAI tools in supporting learning without necessarily measuring their results on the learners. For example, studies by Lewis (2022) and Ouh et al. (2023) focused on validating whether code solutions derived from ChatGPT were correct and efficient, GenAI evaluations (Doughty et al., 2024; Finnie-Ansley et al., 2023), analyzing chat log data from learners (Kazemitabaar et al., 2023b). Their studies shed light on the practical value of GenAI for developing the right solutions, while Roest et al. (2024) investigated the role of GenAI in providing next-step hints and generating code. Wermelinger, (2023) tended to prioritize qualitative insights over quantitative assessment, while Petrovska et al., (2023) employed formative and summative assessments to evaluate the effectiveness of GenAI tools in enhancing learning outcomes. This included analyzing pre-test and post- test intervention (Yilmaz & Yilmaz, 2023b), and peer assessments to quantify improvements in programming skills attributed to using GenAI (Sheese et al., 2024).

The variation in the application of assessment methods highlights a gap in literature in assessing the effects of GenAI tools on student learning and programming skills. Due to the lack of standardize frameworks for assessing how well these tools can promote skills gained after the integration; the integration of GenAI into programming education requires a clear and thorough assessment strategy to measure the contribution such innovative tools make to students' learning and programming skills. Researchers should consider how various assessment strategies complement the pedagogical benefits of GenAI, which would give a complete understanding of its role within programming education.

Integration Process of GenAI in Programming Education

Although assessment methods are related to measuring and evaluating the effectiveness of GenAI tools on student learning and performance, the integration process is vital for advancing educational practices, as it provides a clear framework for replication and assessment of effectiveness. A comprehensive integration process featuring instructor training, technical support, and student orientation creates a clear framework for replication and evaluation while ensuring smooth, interactive engagement between learners and GenAI systems. For instance, Agbo et al., (2019a, 2019b) identified that planned integration enhances student engagement and comprehension for smart learning environments that promote better programming education. Table 6 highlights the importance of the integration process of GenAI in programming education, this clarity enables researchers to identify best practices and adapt approaches that improve students' engagement in making programming more accessible, interactive and enabling educators to measure student understanding and performance over time by generating a variety of assessment types, including multi-choice questions (MCQs) and formative assessments (Doughty et al., 2024; Petrovska et al., 2023; Savelka et al., 2023a). This structured approach provides valuable insights into student learning and opens up replication and validation possibilities for researchers in

Table 6 Importance of integration process of GenAI tools in programming education

Authors	Integration process	Role of integration	Pedagogical impact	Significance rating
Doughty et al., 2024; Petrovska et al., 2023; Savelka et al., 2023a	Generate assessments (MCQs, formative and summative assessments)	Enhances evaluation techniques by providing diverse methods to gauge student understanding and track performance	Establishes robust feedback loops that align assessments with learning goals	Highly Impactful
Kazemitabaar et al., 2023b; Lewis, 2022; Prather et al., 2023	Users interact with Codex for code generation and suggestions	Facilitates hands-on experience with AI tools, promoting active learning, creativity, and confidence in coding	Enhances hands-on learning; however, it requires supplementary traditional methods to secure foundational skills	Moderately Impactful
Jonsson and Tholander, 2022; Sun et al., 2022	Input problem statements and exercise instructions; evaluate generated solutions	Encourages critical thinking and helps students assess AI-generated outputs critically	Encourages reflective learning and strengthens students' analytical skills through validation against expected outcomes	Impactful
Reeves et al., 2023; Rose et al., 2023; Smolansky et al., 2023	Workshops and experimental research processes	Provides structured environments for exploring GenAI applications and fostering collaborative learning	Drives interactive, immersive experiences that lead to lasting skill development	Highly Impactful
Kazemitabaar et al., 2023a; Wu et al., 2023	Students interact with code snippets and solve Parsons problems	Enables practical assessment of programming skills through direct engagement with code examples and systematic problem-solving approaches	Effectively bridges theory and practice by reinforcing key programming concepts through targeted exercises	Impactful
Hou et al., 2024; Sheese et al., 2024; Yilmaz & Yilmaz, 2023b	Learners input descriptions, receive generated code; integrate with online platforms	Bridges the gap between natural language and coding, facilitating accessible learning for diverse student groups	Improves ease of learning; impact may vary depending on AI accuracy and cultural relevance	Moderately Impactful
Berrezueta-Guzman, & Krusche, 2023; Tran et al., 2023a, 2023b	Students receive feedback and submit queries for assistance	Fosters a supportive learning environment by providing timely feedback and encouraging self-directed learning	Reinforces learning through consistent, responsive feedback that helps address student queries in real time	Highly Impactful
Chen et al., 2023; Roest et al., 2024	Analyze the ability to solve programming exercises and compare generated concepts with experts' solutions	Evaluates problem-solving capabilities, helping students to learn from expert demonstrations	Promotes evaluative skills and provides practical benchmarks for further learning improvements	Impactful
Budhiraja et al., 2024; Finnie-Ansley et al., 2023	Prompt engineering for next-step hints and code explanation tools	Develops strategic thinking and comprehension of programming concepts by guiding students in utilizing effective prompts	Enhances critical understanding of AI capabilities when students learn to effectively shape prompts, although success depends on the student's initial familiarity with the concepts	Moderately to Highly Impactful

Table 6 (continued)

Authors	Integration process	Role of integration	Pedagogical impact	Significance rating
Sarkar, 2023; Wermelinger, 2023	Analyze student usage of ChatGPT and evaluate their ability to generate correct solutions	Provides insights into student engagement with AI tools and assesses technical proficiency	Provides useful data for adjusting instructional strategies, with potential for deeper insights upon detailed analysis	Moderately Impactful
Popovici, 2024; Yilmaz & Yilmaz, 2023a	Generate code, explanations, and tests; apply to existing tasks	Encourages innovative thinking and efficient use of AI tools in diverse programming contexts	Strengthens overall creativity and reinforces curriculum objectives when integrated seamlessly into learning activities	Highly Impactful

various contexts. For example, a well-framed integration process improves evaluation techniques, as students are more confident in learning how to code when engaging with GenAI-generated code suggestions (Lewis, 2022; Prather et al., 2023).

Integrating GenAI into programming education has also sparked various opposing viewpoints that merit consideration. A significant concern is the risk of students becoming overly dependent on these tools (Finnie-Ansley et al., 2022; Hoq et al., 2024). This reliance may lead to reduction in understanding programming logic, and higher-order thinking skills. Therefore, educators are encouraged to adopt an approach that ensure that while using GenAI in programming, the higher-order skills and understanding of programming logic is not hindered.

Skills Gained

GenAI can be a dynamic partner in learning rather than simply serving as an automated code generator when strategically implemented. For example, some studies introduced GenAI-assisted code with deliberate bugs or inefficiencies, prompting students to debug. DeLiema et al. (2019) and Hsu (2025) indicated that debugging, as a teaching activity, is an ideal context for fostering the development of critical thinking, reflective thinking, logical thinking, and adaptive thinking skills. Specifically, when learners analyze a GenAI-generated response, they debug by critically identifying and correcting errors or misinterpretations of GenAI responses, thereby aiding multiple brainstorming sessions. Additionally, a significant increase in students' computational thinking skills, programming self-efficacy, and motivation was observed among students who used ChatGPT in a flipped classroom approach combined with hands-on coding activities (Yilmaz & Yilmaz, 2023b). This Highlights that providing exercises with expected outcomes and analyzing ChatGPT-generated solutions (Ouh et al., 2023) improves students' programming skills.

However, understanding how these skills are developed and preserved in the presence of GenAI tools requires consideration, as the skills resulting from integrating GenAI into programming education highly vary depending on the actual teaching methodology, specific GenAI tool use, and assessment methods put in place for such integration, as shown in Fig. 5.

The effectiveness of these tools can be influenced by the teaching methods, because the teaching methods sometimes determine how well the students will come

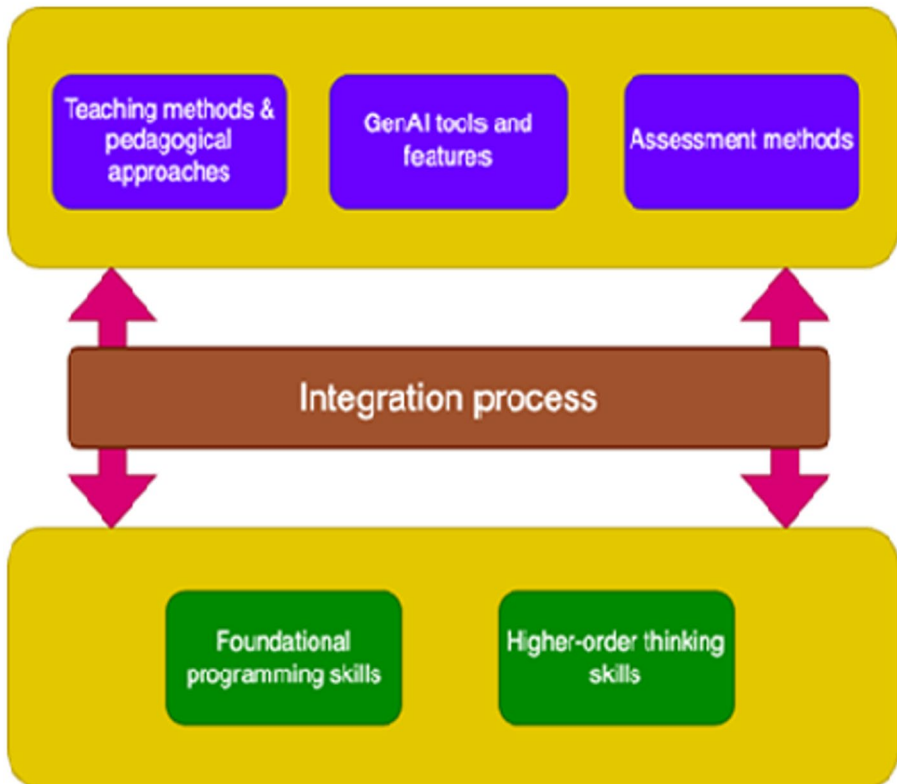


Fig. 5 Alignment of teaching methods, GenAI tools, and assessment strategies in programming education

in contact with the GenAI tool. Choosing the appropriate assessment methods helps to measure the learning outcome, before and after the integration of GenAI tools. Additionally, a structured integration process has the ability to ensure that GenAI supports meaningful, effective, and responsible learning. For example, a hands-on project-based teaching method encourages active learning and problem-solving (Qadir, 2023; Yilmaz & Yilmaz, 2023b), automated next-step hints (Roest et al., 2024) support students when they are stuck in programming exercises, enhancing their problem-solving skills and understanding of programming concepts. Workshops and experimental research processes (Reeves et al., 2023; Rose et al., 2023; Smolansky et al., 2023) provides structured environments for exploring GenAI applications, allowing students to apply GenAI tools in meaningful contexts and enhancing their understanding of foundational programming principles.

However, the results of the study by Kazemitabaar et al., (2023b) showed that students performance varied based on learner engagement levels, with some participants effectively integrating and interacting with the generated code. While others struggled to incorporate it into their solutions. This disparity often traces back to how the tools are introduced, the teaching method, and how they are scaffolded and

supported in the programming classroom. This variability emphasizes the need for structure strategies, including clear usage guidelines, teaching methods, assessment methods, integration processes, and ongoing feedback loop transformation. Hence, to achieve higher-order thinking skills and a solid grasp of fundamental programming logic, it is essential to ensure that all components from the integration process, teaching methods, GenAI tools, and assessment strategies are aligned and purposefully designed. This alignment creates an educational environment where students can engage deeply with content, explore complex problems, and develop resilience in their coding abilities. This rounded approach not only prepares students for the challenges of the programming field but also equips them with the cognitive skills necessary to thrive in an increasingly technology-driven world.

Limitation of the Integrating GenAI into Programming Education

The finding highlights that the seamless integration of GenAI into programming was hindered by several factors, such as the tool's limitation, the teaching method employed, the integration process, and the lack of inclusivity of GenAI tools, as shown in Fig. 6. First, as traditional teaching methods may limit collabora-

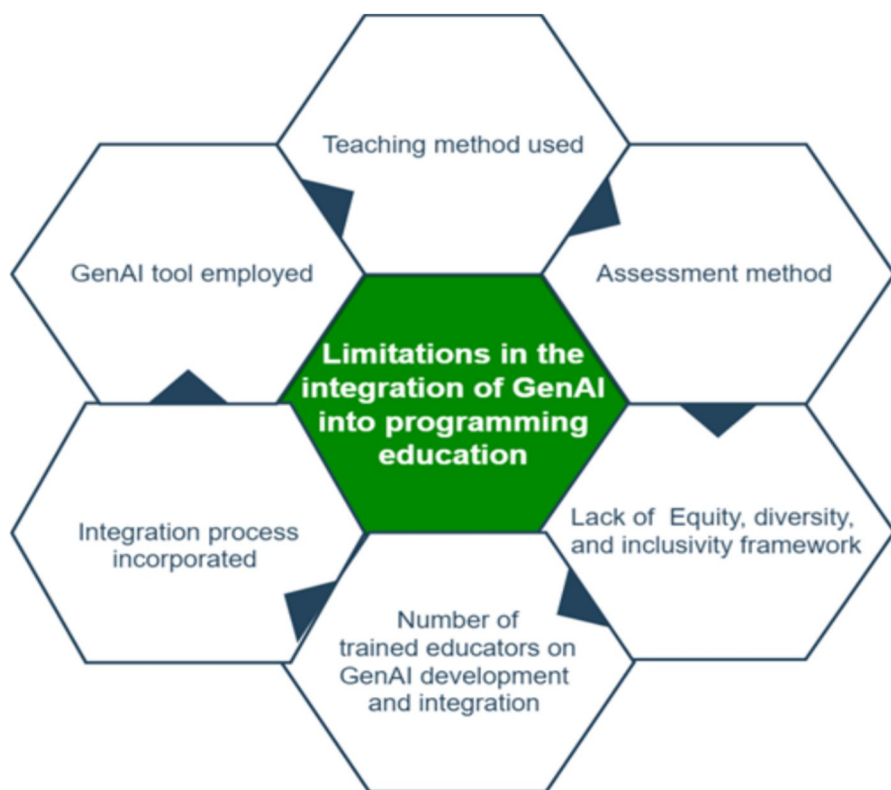


Fig. 6 Limitation found in the integration of GenAI

tive or interactive engagement opportunities that GenAI tools offers. Second, the integration processes used often have limited structure, with many implementations driven more by tool availability than strategic alignment with curriculum objectives or learner needs. Third, students' potential to become overly-reliant on GenAI tools; hindering the development of skills such as higher-orders thinking skill, understanding of programming logic and, lack of inclusivity, and cultural contextualization. Furthermore, there are limited comprehensive framework and assessment methods, particularly those that track long-term skill retention and cognitive development, limiting educators' ability to measure GenAI's impact effectively.

To promote inclusivity through equitable access, it is necessary to incorporate a lens of diversity. Roth et al. (2021) highlight that to be able to incorporate equity, diversity, and inclusion (EDI), an improvement is needed in the universal learning design (UDL) framework. This framework, which emphasizes providing multiple means of engagement, representation, and action or expression, is currently inadequate in addressing inclusivity. This can be achieved by offering culturally relevant examples to all learners, bringing cultural responsiveness and multiple choices regarding how learners interact with content. Woodley et al. (2017) state that although creating culturally responsive online learning happens to be the best practice in utilizing the online classroom environment, such strategies are not always readily available within the traditional learning object delivery model.

Additionally, there is a scarcity of research on programming languages developed with the lexicons of an African indigenous language, unlike those with the lexicons of European and Asian languages (Olatunji et al., 2021). This barrier has made it hard for learners in Africa with limited knowledge of the English language to access resources and learn programming even after a seamless integration of the GenAI tool, which has hindered their mastery of programming concepts. According to Pflapsen (2011), mother tongue computing (MTC) is an effective strategy for improving access to ICT to achieve education for all. Without a robust equity, diversity, and inclusivity (EDI) framework, there is a risk that GenAI resources may not be equitably accessible to all students, particularly those from underrepresented or marginalized backgrounds (Graßl, 2022). Another limitation, according to De La Higuera (2019), is the lack of trained educators, stating that it is impossible to introduce the topic without trained teachers. The lack of training can also lead to educators' resistance to adopting new technologies. As number of trained educators significantly influences the successful integration of AI tools in programming education, as educators play a pivotal role in facilitating student learning and engagement with technology (UNESCO, 2018). These factors collectively affect how well GenAI is integrated into programming education, stressing the need for a balanced method for integrating GenAI into programming education, not as a standalone solution but as a tool whose success depends on considerate implementation, guided use, and alignment with inclusive and reflective teaching practices.

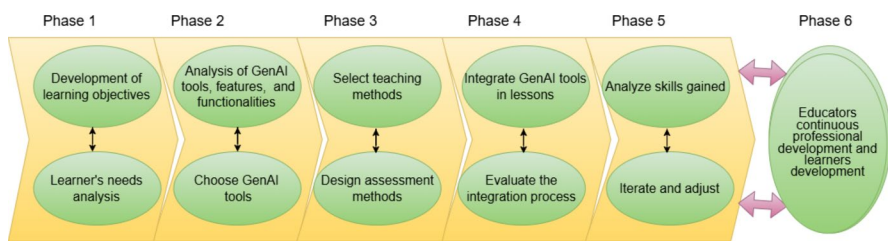


Fig. 7 GenAI-Ped framework

Theoretical Frameworks for Integration of GenAI in Programming Education (GenAI-Ped)

Building on the literature review, this study introduces the GenAI-programming framework for programming education called GenAI-Ped framework (Fig. 7) to guide GenAI integration in programming education and ensure that foundational programming principles and higher-order thinking skills are preserved and enhanced. This framework was adapted from Prasad et al. (2024) self-regulated learning (SRL) framework to ensure that GenAI-Ped included student-driven technique, Bloom's taxonomy (Krathwohl, 2002) and needs analysis (Fayzulloeva & Mustafoeva, 2020) to help with crafting lesson objectives, defining learning goals, and developing instructional activities. The AI-Lab framework (Dickey et al., 2023) was also incorporated to guide prompt-engineering training, and principles of Universal Design for Learning (UDL) (Roth et al., 2021) were added to enhance inclusivity.

Phase 1: Learner's Needs Analysis and Development Learning Objectives

The foundation of effective GenAI integration is based on a thorough understanding of learners' diverse needs. This phase begins with a comprehensive analysis of student demographics, prior programming knowledge, and accessibility requirements, where educators and students jointly identify learning priorities through surveys, assessments, focus groups, or self-assessment reflections. Learners articulate their programming backgrounds, accessibility requirements, and personal goals (mastering debugging vs. creative coding). Learner needs analysis is crucial before defining learning objectives, designing assessments, structuring the course, and developing materials (Sari et al., 2020). For example, students might propose objectives like "Use GenAI to generate three code variants for a problem, then critically compare efficiency trade-offs." Educators collaborate with institutional stakeholders to map these insights to SMART (Specific, Measurable, Achievable, Relevant, Time-bound) learning objectives grounded in Bloom's Taxonomy, ensuring alignment with higher-order thinking skills such as critical analysis and creative problem-solving.

Phase 2: Analysis of GenAI Tools Features and Functionalities

With learning objectives established, educators evaluate each tool to determine its strengths and limitations while involving students in the process. For instance, stu-

dents document their perception of the tool in reflective logs; actively test and critique GenAI tools and document tool strengths and limitations through reflective journals. For instance, learners might note: “Copilot accelerated my loop design but failed to explain algorithmic logic.” Understanding a GenAI tool’s core components, perception, and use cases is essential to seamlessly integrate it into programming education. Chiu et al. (2024) suggest that all students should understand AI history and contemporary advancements such as the “fourth industrial revolution”. Students must not only learn to use GenAI but also understand its features and limitations, since this knowledge enables them to craft effective prompts and critically evaluate AI-generated code (Fernandez & Cornell, 2024). Since GenAI is changing the fundamental concept of how machines work (Zhang & Lu, 2021), mastering the essential knowledge of GenAI, helping them recognize how GenAI analyzes data, learns from data, the technical skills, perception, and the social impact of GenAI. These insights guide educators in selecting tools that balance technical utility with transparency and explainability, prioritizing tools that empower learners to interrogate outputs rather than passively accept them.

Next, educators should select and configure GenAI tools that offer accessibility features such as screen readers, text-to-speech, and customizable interfaces to ensure equitable use by all students. Secondly, exploring the tool allows user-defined settings and parameters to personalize support for diverse learners. Thirdly, choosing tools that explain code suggestions, auto-completions, or generated code, fostering understanding beyond mere output. Finally, evaluating the biases within GenAI tools helps to select the right GenAI that suit the learning objectives and learners needs.

Phase 3: Teaching Methods, Assessment Methods, and Prompt Engineering

This phase bridges theoretical objectives with actionable teaching strategies, assessment method, and prompt engineering training, ensuring GenAI tools enhance programming skills rather than replace them. This phase requires selecting teaching methods that best meet instructional objectives and actively engage learners. For instance, educators could design hybrid pedagogical models, blending traditional methods (Code exercises to ensure hands-on learning, in-class programming exercises) with AI-driven activities (GenAI-assisted debugging programming challenges) to provide a robust framework for programming education. This might begin with live code-along demonstrations in which the instructor and the class co-develop a function using ChatGPT’s suggestions, pausing to critique the output and to refine prompts in real time. By weaving these methods together, educators develop both programming logic and the higher order thinking skills needed to craft, critique, and adapt prompts effectively.

Assessment methods should be designed not only to evaluate technical competence but also to assess students’ evolving understanding by integrating both formative and summative approaches. For instance, Formative assessment, embedded throughout the instructional sequence, may take the form of mini-quizzes, assignments, draft code submissions, brief programming tasks, or structured reflective journals (students’ reflections logs collected through written narratives and guided discussions, and GenAI chat logs). These ongoing assessment yield immediate feedback that instructors can use to address misconceptions and scaffold support before errors become

deep-rooted. Summative assessment takes place at the conclusion, it includes end of module examinations, capstone programming projects, and curated GenAI interaction logs to evaluate whether students have met the defined learning objectives. These assessments help to gauge technical proficiency and reflect improvements in higher-order thinking skills and understanding of programming logic.

In the prompt-engineering module, students practice writing clear, precise prompts to elicit accurate and useful AI responses. This session is designed to be interactive, with students discussing their methods in small groups and receiving immediate feedback on their approach. In this phase, students are guided to design prompts following the plan-monitor-reflect cycle proposed by Prasad et al., (2024). For example: Plan: *“I need a function to sort arrays. How should I phrase my prompt?”* Monitor: *“Why did Codex generate an inefficient algorithm? Let me adjust my prompt.”* Reflect: *“Did refining prompts improve my grasp of sorting logic?”* This active participation helps both teachers and learners understand the intricacies of how these tools function, including how variations in prompts can affect the quality of the output. Moreover, meaningful prompting provides a foundation for engagement, motivating learners to abstract key elements, decompose complex problems, and logically deal with each more minor problem (Denny et al., 2024), building confidence and enabling a sense of ownership over the technology, ensuring learners become active participants rather than passive users, further motivating students' interaction with GenAI (Fernandez & Cornell, 2024).

Phase 4: Integrate GenAI Tool for the Lesson

Integrating GenAI tools into classroom activities is executed with students taking a leading role. Instructors act more as facilitators by offering guidance when necessary, rather than dictating every step. Lessons are developed around real-world programming scenarios where students must interact directly with the GenAI software. They are encouraged to design, test, and adjust their prompts while working on coding challenges individually or in small groups. This hands-on experience provides a richer understanding of both the practical application of programming concepts and the capabilities of GenAI tools. The goal is for students to experience both the creative and evaluative aspects of programming, thereby maintaining a balance between traditional learning and innovative, technology-driven practices. For instance, students could use ChatGPT to debug loops in a Python programming course. They can iteratively refine prompts to identify logic errors and then compare AI-generated codes with peer solutions. Students document their problem-solving journey in digital portfolios, highlighting instances where GenAI enhanced or hindered their understanding. To reduce overreliance on GenAI tool, the integration should be grounded in constructivist learning theories; Vygotsky's Zone of Proximal Development (ZPD) (Shabani et al., 2010). Educators should scaffold the integration process to provide intensive initial support; where students make extensive use of GenAI, receiving substantial guidance from teachers and GenAI initially, and assistance is gradually reduced as students progress. Scaffolded fading of GenAI support draws on Vygotsky's ZPD, empowering students to transition from guided GenAI use to independent critical evaluation, thereby fostering ownership of learning. This intentional

fading of assistance helps students develop higher-order thinking skills as they learn to analyze and evaluate the outputs independently. Furthermore, continuous assessment of integration process is necessary, as it helps recognize, address, and rectify biases (Jackson, 2024), this will be done in phase 5.

Phase 5: Analyze Sills Gained, Iterate and Adjust

Analyzing the skills helps inform educators about the effectiveness of the teaching methods, assessment designs, integration process and the extent to which scaffolding has helped students gain independence. For instance, educators can create a rubric for quantitative data (pre/post-test scores, programming project) to evaluate students higher-order thinking skills and understanding of programming logic; adjust the weights and criteria to suit and their specific needs and requirement. While also using an inductive coding approach to analyze qualitative data (reflective journal). This analysis will help reveals whether teaching method, integration process or GenAI tool succeeded or missed the mark. Simultaneously, learner feedback and result are then channeled to Phase 6 because the findings may often require adjustments to educator training, student support frameworks, or tool configurations. This helps in refining learning objectives, modification of teaching method, or integration process, reanalyzing GenAI tools features and functionalities to improve alignment between pedagogy, technology, and learner needs. And creating a dynamic feedback loop that ensures the framework evolves with emerging student needs and technological advancements.

Phase 6: Educators Continuous Professional Development and Learners' Development

In the final phase, the focus shifts to continuous development for both educators and students. Recognizing that both groups must adapt to the evolving nature of GenAI tools. For educators, these sessions provide updates on new advancements in GenAI applications and effective methods to support student growth. Kong et al. (2020) and Fundi et al. (2024) highlights that implementing a teacher development program helps to enhance teachers' capability to develop computational thinking in students through a programming curriculum. As teachers' perspectives are essential to improving programming performance amongst the student population because literacy is about knowing and applying knowledge beneficially (Chiu et al., 2024). For students, the session offers continued practice in prompt engineering, debugging, and analyzing GenAI outputs. The training is designed to be practical and accessible, with a steady stream of collaboration and shared learning opportunities. This ongoing training reinforces the commitment to creating a classroom environment where technological integration is smoothly combined with a strong emphasis on individual learning and professional growth.

Comparison GenAI-Ped Framework with Existing Framework

The Self-Regulated Learning Framework (SRL) by Prasad et al., (2024) treats generative AI as a “pair thinking” partner. It supports planning, monitoring, control, and

reflection to help novices avoid overreliance through explicit metacognitive strategies. GenAI Ped adopts SRL in Phase 1 with self-assessment surveys and in Phase 3 with guided reflections. It then adds accessible tool settings, Universal Design for Learning, and hands on prompt engineering training and scaffolding techniques. Thereby offering a more holistic, institution ready blueprint that ties SRL strategies to curriculum objectives, assessment metrics, and ongoing educator development. Su and Yang's (2023) IDEE cycle (Identify outcomes, Determine automation, Ensure ethics, Evaluate impact) outlines planning steps but offers no guidance for classroom use. GenAI Ped aligns learner needs with Bloom's objectives in Phase 1, evaluates tools for accessibility and bias in Phase 2, embeds ethics and reflection in Phases 3 and 4, and sustains continuous evaluation and educator development in Phases 5 and 6. Dickey et al. (2023) AI-Lab framework provides valuable guidance in introducing students to tools like GitHub Copilot and ChatGPT through prompt workshops and perception surveys. However, it leaves open the question of how those activities directly map onto measurable learning outcomes. GenAI Ped transforms isolated labs into a comprehensive, human centered program that fosters equitable, inclusive, and lasting integration of generative AI throughout the student learning journey. Table 7 shows the comparison of GenAI-Ped framework with existing framework.

Conclusion

While GenAI tools like ChatGPT and Codex streamline code generation and foster engagement, they can hinder the development of higher-order thinking skills and understanding of programming logic if applied without structured learning activities. Other limitations that hinder the seamless integration include accessibility barriers, cultural biases, and educator readiness, emphasizing the necessity of inclusive policies and professional development.

The proposed GenAI-Ped framework addresses these gaps by ensuring these tools do not replace higher-order thinking skills and understanding of programming logic by guiding educators first to understand students' needs, like skill levels and acces-

Table 7 Comparison of GenAI-Ped framework with existing framework

Framework	Objective	Focus areas	Pedagogical strategies	Strengths	Key limitation	Difference vs GenAI-Ped
GenAI-Ped	Integrate GenAI equitably, retain skills	Equity, curriculum alignment	UDL, Bloom's Taxonomy, Reflection	Holistic, inclusive	Resource-intensive	NA
SRL (Prasad et al., 2024)	Foster self-regulation with GenAI	Metacognition, iterative tasks	Reflection, code adaptation	Deep cognitive engagement	No equity focus	Less structured; no equity/skill alignment
AI-Lab (Dickey et al., 2023)	Mitigate GenAI over-reliance	Error detection, prompts	Lab-based error analysis	Practical error critique	Narrow technical scope	No equity/long-term focus

sibility challenges, and then carefully select GenAI tools that align with course goals. Instead of letting students passively accept GenAI-generated code, the framework encourages active engagement: Learners practice refining prompts, compare GenAI solutions, and gradually reduce tool dependency as their skills grow. By blending GenAI's efficiency with hands-on practices, GenAI-Ped helps students build a deeper understanding of programming logic and higher-order thinking skills.

However, while GenAI-Ped offers a structured approach from an objective setting to evaluation, it still requires empirical validation through the classroom to confirm its broad applicability. Future research will involve piloting the framework in introductory programming courses to assess its impact on learning outcomes following the GenAI-Ped framework implementation process shown in Appendix 1. It would prioritize mixed-methods studies that involve a within-group experimental design, pre-and post-tests to measure technical proficiency (for instance, higher-order thinking skills and understanding of programming logic), and qualitative analysis of student reflections on GenAI interactions.

Appendix 1

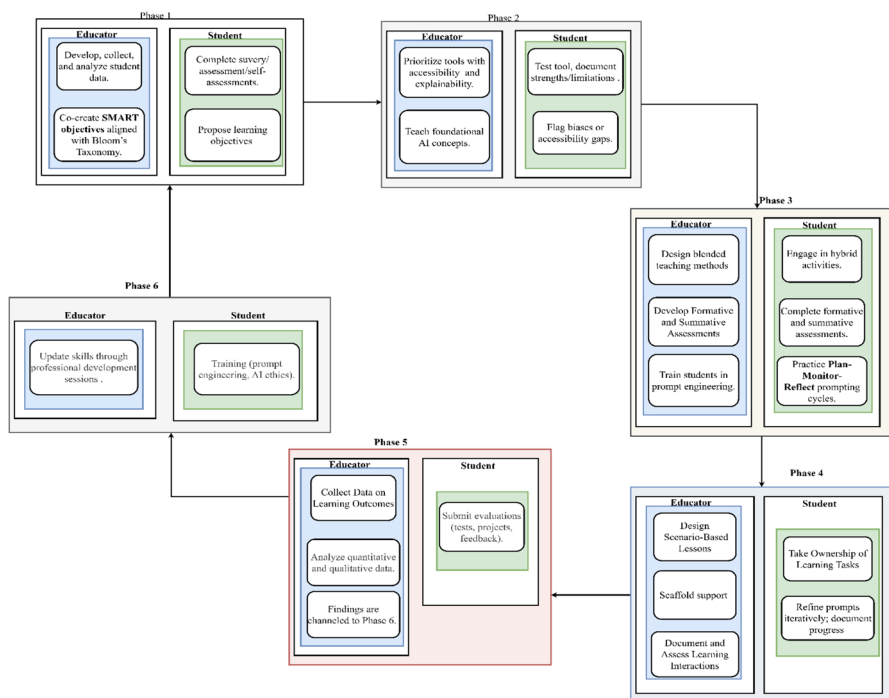


Fig. 8 Implementation process of GenAI-Ped framework

Author's Contribution 1.Jemimah Nathaniel: Conducted the literature review, drafted the manuscript; and synthesized findings in alignment with my PhD thesis objectives. 2.Solomon Sunday Oyelere (Supervisor 1): Provided guidance on research questions, research design and methodology; contributed to the interpretation of results; Offered expertise in the theoretical framework, and offered feedback on manuscript drafts. 3.Jarkko Suhonen (Supervisor 2): Assisted in refining research questions; ensured the study's relevance to programming education; and contributed to the review and revision of the manuscript. 4. Matti Tedre (Supervisor 3): contributed to the final revisions of the manuscript.

Funding Open access funding provided by University of Eastern Finland (including Kuopio University Hospital). Not applicable.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Abosalem, Y. (2016). Assessment techniques and students' higher-order thinking skills. *International Journal of Secondary Education*, 4(1), 1–11. <https://doi.org/10.11648/j.ijssedu.20160401.11>
- Agbo, F. J., Oyelere, S. S., Suhonen, J., & Adewumi, S. (2019a). A systematic review of computational thinking approach for programming education in higher education institutions. In *Proceedings of the 19th Koli calling international conference on computing education research* (pp. 1–10). <https://doi.org/10.1145/3364510.3364521>
- Agbo, F. J., Oyelere, S. S., Suhonen, J., & Tukiainen, M. (2019b). Identifying potential design features of a smart learning environment for programming education in Nigeria. *International Journal of Learning Technology*, 14(4), 331–354. <https://doi.org/10.1504/IJLT.2019.106551>
- Anagnostopoulos, S. (2023). ChatGPT impacts in programming education: A recent literature overview that debates ChatGPT responses. In *Cultural Technology and Communication, University of the Aegean, Mytilene, Lesbos*, 81100. <https://doi.org/10.12688/f1000research.141958.1>
- Bermejo, M. M., Herranz, S. M., Valladolid, R. C., Gómez, D. C., Capelo, I. C., Rodríguez-García, M. A., ... & Carrillo, M. G. (2024). Teaching programming using generative ai in the classroom. In *ICERI2024 Proceedings* (pp. 7412–7418). IATED. <https://doi.org/10.21125/iceri.2024.1786>
- Berrezueta-Guzman, J., & Krusche, S. (2023). Recommendations to create programming exercises to overcome ChatGPT. In *2023 IEEE 35th International Conference on Software Engineering Education and Training (CSEE&T)* (pp. 147–151). <https://doi.org/10.35542/osf.io/expuq>
- Bomah, K. B. (2014). Digital divide: effects on education development in Africa. In Conference in LYIT Dept. of Computing: Technical Writing Presentation. Letterkenny Institute of Technology. www.researchgate.net/publication/275350414
- Božić, V. (2023). Artificial intelligence as the reason and the solution of digital divide. *Language Education and Technology*, 3(2). <https://doi.org/10.13140/RG.2.2.10494.66880>

- Budhiraja, R., Joshi, I., Challa, J. S., Akolekar, H. D., & Kumar, D. (2024). “It’s not like Jarvis, but it’s pretty close!”—Examining ChatGPT’s usage among undergraduate students in computer science. In *Proceedings of the 26th Australasian Computing Education Conference* (pp. 124–133). <https://doi.org/10.1145/3636243.3636257>
- Buitrago, F. F., Casallas, R., Hernández, M., Reyes, A., Restrepo, S., & Danies, G. (2017). Changing a generation’s way of thinking: Teaching computational thinking through programming. *Review of Educational Research*, 87(4), 834–860. <https://doi.org/10.3102/0034654317710096>
- Chaudhry, M. A., & Kazim, E. (2022). Artificial intelligence in education (AIED): A high-level academic and industry note 2021. *AI and Ethics*, 2(1), 157–165. <https://doi.org/10.1007/s43681-021-00074-z>
- Chen, E., Huang, R., Chen, H. S., Tseng, Y. H., & Li, L. Y. (2023). GPTutor: a ChatGPT-powered programming tool for code explanation. In *International Conference on Artificial Intelligence in Education* (pp. 321–327). Springer Nature Switzerland. <https://doi.org/10.1007/978-3-031-36336-8>
- Chiu, T. K., Ahmad, Z., Ismailov, M., & Sanusi, I. T. (2024). What are artificial intelligence literacy and competency? A comprehensive framework to support them. *Computers and Education Open*, 6, 100171. <https://doi.org/10.1016/j.caco.2024.100171>
- Daungcharone, K., Thongkoo, K., Thanyaphongphat, J., & Panjaburee, P. (2023). Evaluating learning motivation and achievement in ubiquitous online collaborative learning among different nationalities. In *Proceedings of the 2023 7th International Conference on Education and E-Learning* (pp. 66–72). <https://doi.org/10.1145/3637989.3638001>
- De La Higuera, C. (2019). A report about education, training teachers and learning artificial intelligence: overview of key issues. *Education, Computer Science*, 1–11. <https://www.TeachingAI-report09072019>
- DeLiema, D., Dahn, M., Flood, V. J., Asuncion, A., Abrahamson, D., Enyedy, N., & Steen, F. (2019). Debugging as a context for fostering reflection on critical thinking and emotion. Deeper learning dialogic learning and critical thinking: Research-based strategies for the classroom. In *Hrsg. von Emmanuel Manalo*, (pp. 209–228). <https://doi.org/10.4324/9780429323058-13>
- Denny, P., Leinonen, J., Prather, J., Luxton-Reilly, A., Amarouche, T., Becker, B. A., & Reeves, B. N. (2024). Prompt problems: A new programming exercise for the Generative AI era. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (pp. 296–302). <https://doi.org/10.1145/3626252.3630909>
- Di, W., Danxia, X., & Chun, L. (2019). The effects of learner factors on higher-order thinking in the smart classroom environment. *Journal of Computers in Education*, 6, 483–498. <https://doi.org/10.1007/s40692-019-00146-4>
- Dickey, E., Bejarano, A., & Garg, C. (2023). Innovating computer programming pedagogy: The AI-Lab framework for generative AI Adoption. <https://doi.org/10.1007/s42979-024-03074-y>
- Doughty, J., Wan, Z., Bompelli, A., Qayum, J., Wang, T., Zhang, J., & Sakr, M. (2024). A comparative study of AI-generated (GPT-4) and human-crafted MCQs in programming education. In *Proceedings of the 26th Australasian Computing Education Conference* (pp. 114–123). <https://doi.org/10.1145/3636243.3636256>
- Fayzulloeva, G. C., & Mustafoeva, N. U. O. A. (2020). A learner needs analysis report. *International Journal of Innovative Research and Development*, 11, 726.
- Fernandez, A. S., & Cornell, K. A. (2024). CS1 with a Side of AI: Teaching software verification for secure code in the era of Generative AI. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (pp. 345–351). <https://doi.org/10.1145/3626252.3630817>
- Finnie-Ansley, J., Denny, P., Becker, B. A., Luxton-Reilly, A., & Prather, J. (2022). The robots are coming: Exploring the implications of OpenAI codex on introductory programming. In *ACM International Conference Proceeding Series* (pp. 10–19). <https://doi.org/10.1145/3511861.3511863>
- Finnie-Ansley, J., Denny, P., Luxton-Reilly, A., Santos, E. A., Prather, J., & Becker, B. A. (2023). My AI wants to know if this will be on the exam: Testing OpenAI’s codex on CS2 programming exercises. In *Proceedings of the 25th Australasian Computing Education Conference* (pp. 97–104). <https://doi.org/10.1145/3576123.3576134>
- Fundi, M., Sanusi, I. T., Oyelere, S. S., & Ayere, M. (2024). Advancing AI education: Assessing Kenyan in-service teachers’ preparedness for integrating artificial intelligence in competence-based curriculum. *Computers in Human Behavior Reports*, 14, 100412.
- Graßl, I. (2022). Diversity in programming education: help underrepresented groups learn programming. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (pp. 272–274). <https://doi.org/10.1109/ICSE-Companion55297.2022.9793790>

- Hoq, M., Shi, Y., Leinonen, J., Babalola, D., Lynch, C., Price, T., & Akram, B. (2024). Detecting ChatGPT-generated code submissions in a cs1 course using machine learning models. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (pp. 526–532). <https://doi.org/10.1145/3626252.3630826>
- Hou, I., Mettelle, S., Man, O., Li, Z., Zastudil, C., & MacNeil, S. (2024). The effects of Generative AI on computing students' help-seeking preferences. In *Proceedings of the 26th Australasian Computing Education Conference* (pp. 39–48). <https://doi.org/10.1145/3636243.3636248>
- Hsu, H. P. (2025). From programming to prompting: Developing computational thinking through large language model-based generative artificial intelligence. *TechTrends*. <https://doi.org/10.1007/s11528-025-01052-6>
- Ibarra-Torres, F., Caiza, G., García, M. V., & Barona-Pico, V. (2024). Use of basic programming tools to foster programming logic in university students with school preparation other than computer science. *Procedia Computer Science*, 237, 413–419. <https://doi.org/10.1016/j.procs.2024.05.122>
- Jackson, A. E. (2024). The evolution of artificial intelligence: A theoretical review of its impact on teaching and learning in the digital age. ZBW – Leibniz Information Centre for Economics, Kiel, Hamburg. <https://hdl.handle.net/10419/280893>
- Jonsson, M., & Tholander, J. (2022). Cracking the code: Co-coding with AI in creative programming education. In *Proceedings of the 14th Conference on Creativity and Cognition* (pp. 5–14). <https://doi.org/10.1145/3527927.3532801>
- Kanika, Chakraverty, S., & Chakraborty, P. (2020). Tools and techniques for teaching computer programming: A review. *Journal of Educational Technology Systems*, 49(2), 170–198. <https://doi.org/10.1177/0047239520926971>
- Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023a). Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (pp. 1–23). <https://doi.org/10.1145/3544548.3580919>
- Kazemitabaar, M., Hou, X., Henley, A., Ericson, B. J., Weintrop, D., & Grossman, T. (2023b). How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. In *Proceedings of the 23rd Koli calling international conference on computing education research* (pp. 1–12). <https://doi.org/10.48550/arXiv.2309.14049>
- Kitchenham, B., & Brereton, P. (2013). A systematic review of systematic review process research in software engineering. *Information and Software Technology*, 55(12), 2049–2075. <https://doi.org/10.1016/j.infsof.2013.07.010>
- Kong, S. C., Lai, M., & Sun, D. (2020). Teacher development in computational thinking: Design and learning outcomes of programming concepts, practices and pedagogy. *Computers & Education*, 151, 103872. <https://doi.org/10.1016/j.compedu.2020.103872>
- Konijn, E. A., & Hoorn, J. F. (2020). Robot tutor and pupils' educational ability: Teaching the times tables. *Computers & Education*, 157, 103970. <https://doi.org/10.1016/j.compedu.2020.103970>
- Kothari, S. (2023). Top generative AI tools: Boost your creativity. www.simplilearn.com/tutorials/artificial-intelligence-tutorial/top-generative-ai-tools
- Krathwohl, D. R. (2002). A revision of Bloom's taxonomy: An overview. *Theory Into Practice*, 41(4), 212–218. <https://doi.org/10.1207/s15430421tip4104>
- Lau, S., & Guo, P. (2023). From "Ban it till we understand it" to "Resistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research* (pp. 106–121). <https://doi.org/10.1145/3568813.3600138>
- Lewis, C. (2022). Automatic programming and education. In *Companion Proceedings of the 6th International Conference on the Art, Science, and Engineering of Programming* (pp. 70–80). <https://doi.org/10.1145/3532512.3539664>
- Li, W., Huang, J. Y., Liu, C. Y., Tseng, J. C., & Wang, S. P. (2023). A study on the relationship between student' learning engagements and higher-order thinking skills in programming learning. *Thinking Skills and Creativity*, 49, Article 101369. <https://doi.org/10.1016/j.tsc.2023.101369>
- Liu, M., Zhang, L. J., & Biebricher, C. (2024). Investigating students' cognitive processes in generative AI-assisted digital multimodal composing and traditional writing. *Computers & Education*, 211, 104977. <https://doi.org/10.1016/j.compedu.2023.104977>
- Loes, C. N., & Pascarella, E. T. (2017). Collaborative learning and critical thinking: Testing the link. *The Journal of Higher Education*, 88(5), 726–753. <https://doi.org/10.1080/00221546.2017.1291257>

- McDougall, A., Murnane, J. S., & Wills, S. (2014). The educational programming language logo: its nature and its use in Australia. In *Reflections on the History of Computers in Education: Early Use of Computers and Teaching about Computing in Schools* (pp. 394–407). https://doi.org/10.1007/978-3-642-55119-2_28
- Merrick, K. E. (2010). An empirical evaluation of puzzle-based learning as an interest approach for teaching introductory computer science. *IEEE Transactions on Education*, 53(4), 677–680. <https://doi.org/10.1109/TE.2009.2039217>
- Moher, D., Liberati, A., Tetzlaff, J., Altman, D. G., Prisma Group. (2010). Preferred reporting items for systematic reviews and meta-analyses: The PRISMA statement. *International journal of Surgery*, 8(5), 336–341. <https://doi.org/10.7326/0003-4819-151-4-20090818000135>
- Motlagh, N. Y., Khajavi, M., Sharifi, A., & Ahmadi, M. (2023). The impact of Artificial intelligence on the evolution of digital education: A comparative study of OpenAI text generation tools including ChatGPT. *Bing Chat, Bard, and Ernie*. <https://doi.org/10.1016/j.jss.2023.111734>
- Nwana, H. S. (1990). Intelligent tutoring systems: An overview. *Artificial Intelligence Review*, 4(4), 251–277.
- Olatunji, E. K., Oladosu, J. B., Odejebi, O. A., & Olabiyisi, S. O. (2021). Design and implementation of an African native language-based programming language. *International Journal of Advances in Applied Sciences*, 10(2), 171–177. <https://doi.org/10.11591/ijaas.v10.i2.pp171-177>
- Ouh, E. L., Gan, B. K. S., Jin Shim, K., & Wlodkowski, S. (2023). ChatGPT, Can you generate solutions for my coding exercises? An evaluation on its effectiveness in an undergraduate java programming course. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1* (pp. 54–60). <https://doi.org/10.1145/3587102.3588794>
- Oyelere, S. S., Suhonen, J., & Laine, T. H. (2017). Integrating parson's programming puzzles into a game-based mobile learning application. In *Proceedings of the 17th Koli Calling International Conference on Computing Education Research* (pp. 158–162). <https://doi.org/10.1145/3141880.3141882>
- Oyelere, S. S., Agbo, F. J., Sanusi, I. T., Yunusa, A. A., & Sunday, K. (2019). Impact of puzzle-based learning technique for programming education in Nigeria context. In *2019 IEEE 19th International Conference on Advanced Learning Technologies (ICALT)* (Vol. 2161, pp. 239–241). <https://doi.org/10.1109/ICCSA>
- Page, M. J., Moher, D., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., Mcdonald, S., ... BMJ Publishing Group. (2021). PRISMA 2020 explanation and elaboration: Updated guidance and exemplars for reporting systematic reviews. *BMJ (Clinical Research Ed.)*. <https://doi.org/10.1136/bmj.n160>
- Palts, T., & Pedaste, M. (2020). A model for developing computational thinking skills. *Informatics in Education*, 19(1), 113–128. <https://doi.org/10.15388/INFEDU.2020.06>
- Pereira, F. D., Pires, F., Fonseca, S. C., Oliveira, E. H., Carvalho, L. S., Oliveira, D. B., & Cristea, A. I. (2021). Towards a human-AI hybrid system for categorising programming problems. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education* (pp. 94–100). <https://doi.org/10.1145/3408877.3432422>
- Petrovska, O., Clift, L., & Moller, F. (2023). Generative AI in software development education: Insights from a degree apprenticeship programme. In *Proceedings of the 2023 Conference on United Kingdom & Ireland Computing Education Research* (pp. 1–1). <https://doi.org/10.1145/3610969.3611132>
- Pflepsen, A. (2011). Improving learning outcomes through mother tongue-based education. Retrieved from RTI International, May, 10, 2013. <https://www.rti.org/brochures/improving-learning-outcomes-through-mother-tongue-based-education>
- Phung, T., Pădurean, V. A., Cambronero, J., Gulwani, S., Kohn, T., Majumdar, R., & Soares, G. (2023). Generative AI for programming education: benchmarking ChatGPT, GPT-4, and human tutors. In *Proceedings of the 2023 ACM Conference on International Computing Education Research* (pp. 41–42). <https://doi.org/10.1145/3568812.3603476>
- Popat, S., & Starkey, L. (2019). Learning to code or coding to learn? A systematic review. *Computers & Education*, 128, 365–376. <https://doi.org/10.1016/j.compedu.2018.10.005>
- Popovici, M. D. (2024). ChatGPT in the classroom. Exploring its potential and limitations in a functional programming course. *International Journal of Human-Computer Interaction*, 40(22), 7743–7754. <https://doi.org/10.1080/10447318.2023.2269006>
- Prasad, P., & Sane, A. (2024, March). A self-regulated learning framework using generative AI and its application in CS educational intervention design. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (pp. 1070–1076). <https://doi.org/10.1145/3626252.3630828>

- Prather, J., Denny, P., Leinonen, J., Becker, B. A., Albluwi, I., Craig, M., Keuning, H., Kiesler, N., Kohn, T., Luxton-Reilly, A., MacNeil, S., Petersen, A., Pettit, R., Reeves, B. N., & Savelka, J. (2023). The robots are here: navigating the generative AI revolution in computing education. In *Proceedings of the 2023 working group reports on innovation and technology in computer science education* (pp. 108–159). <https://doi.org/10.1145/3623762.3633499>
- Qadir, J. (2023). Engineering education in the era of ChatGPT: Promise and pitfalls of generative AI for education. In *2023 IEEE global engineering education conference (EDUCON)* (pp. 1–9). <https://doi.org/10.1109/EDUCON54358.2023.10125121>
- Ragonis, N., & Shmallo, R. (2022). The application of higher-order cognitive thinking skills to promote students' understanding of the use of static in object-oriented programming. *Informatics in Education*, 21(2), 331–352. <https://doi.org/10.15388/infedu.2022.10>
- Reeves, B., Sarsa, S., Prather, J., Denny, P., Becker, B. A., Hellas, A., Leinonen, J. (2023). Evaluating the performance of code generation models for solving parsons problems with small prompt variations. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1* (pp. 299–305). <https://doi.org/10.1145/3587102.3588805>
- Reiche, M., & Leidner, J. L. (2023). Bridging the programming skill gap with ChatGPT: a machine learning project with business students. In *European Conference on Artificial Intelligence* (pp. 439–446). Springer Nature Switzerland. <https://doi.org/10.1007/9783-031-50485-3>
- Roest, L., Keuning, H., & Jeuring, J. (2024). Next-step hint generation for introductory programming using large language models. In *Proceedings of the 26th Australasian Computing Education Conference* (pp. 144–153). <https://doi.org/10.48550/arXiv.2312.10055>
- Rose, K., Massey, V., Marshall, B., & Cardon, P. (2023). Is professors' perspectives on AI -assisted programming. *Issues in Information Systems*, 24(2), 178–190. <https://doi.org/10.48009/2iis2023115>
- Roth, A., Singh, G., & Turnbow, D. (2021). Equitable but not diverse: Universal design for learning is not enough. *Open Access Publications from the University of California*. <https://www.inthelibrarywiththeleadpipe.org/2021/equitable-but-not-diverse/>
- Sallam, M. (2023). The utility of ChatGPT as an example of large language models in healthcare education, research and practice: Systematic review on the future perspectives and potential limitations. *MedRxiv*, 2023–02. <https://doi.org/10.3390/healthcare11060887>
- Santos, S. C., Tedesco, P. A., Borba, M., & Brito, M. (2020). Innovative approaches in teaching programming: A systematic literature review. In *Proceedings of the 12th International Conference on Computer Supported Education* (Vol. 1, pp. 205–214). <https://doi.org/10.5220/0009190502050214>
- Sari, Y. I. H., Wienanda, W. K., & Nugraheni, N. E. (2020). Needs analysis to develop teaching materials at vocational college UGM. *Jurnal Pendidikan Vokasi*, 10(2), 138–149. <https://doi.org/10.21831/jp.v.v10i2.27934>
- Sarkar, A. (2023). Will code remain a relevant user interface for end-user programming with Generative AI models? In *Proceedings of the 2023 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (pp. 153–167). <https://doi.org/10.1145/3622758.3622882>
- Sarsa, S., Denny, P., Hellas, A., & Leinonen, J. (2022). Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research* (pp. 27–43). <https://doi.org/10.1145/3501385.3543957>
- Savelka, J., Agarwal, A., An, M., Bogart, C., & Sakr, M. (2023a). Thrilled by your progress! Large language models (GPT-4) no longer struggle to pass assessments in higher education programming courses. In *Proceedings of the 2023 ACM Conference on International Computing Education Research* (pp. 78–92). <https://doi.org/10.1145/3568813.3600142>
- Savelka, J., Agarwal, A., Bogart, C., Song, Y., & Sakr, M. (2023b). Can generative pre-trained transformers (GPT) pass assessments in higher education programming courses?. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education* (pp. 117–123). <https://doi.org/10.1145/3587102.3588792>
- Scherer, R., Siddiq, F., & Viveros, B. S. (2020). A meta-analysis of teaching and learning computer programming: Effective instructional approaches and conditions. *Computers in Human Behavior*, 109, 106349. <https://doi.org/10.1016/j.chb.2020.106349>
- Setiawan, A., Malik, A., Suhandi, A., & Permanasari, A. (2018). Effect of higher order thinking laboratory on the improvement of critical and creative thinking skills. In *IOP Conference Series: Materials Science and Engineering* (Vol. 306, No. 1, p. 012008). IOP Publishing. <https://doi.org/10.1088/1757-899X/306/1/012008>

- Shabani, K., Khatib, M., & Ebadi, S. (2010). Vygotsky's zone of proximal development: Instructional implications and teachers' professional development. *English Language Teaching*, 3(4), 237–248. <https://doi.org/10.5539/elt.v3n4p237>
- Sheese, B., Liffiton, M., Savelka, J., & Denny, P. (2024). Patterns of student help-seeking when using a large language model-powered programming assistant. In *Proceedings of the 26th Australasian computing education conference* (pp. 49–57). <https://doi.org/10.1145/3636243.3636249>
- Singla, A. (2023). Evaluating ChatGPT and GPT-4 for visual programming. In *Proceedings of the 2023 ACM Conference on International Computing Education Research* (pp. 14–15). <https://doi.org/10.1145/3568812.360347>
- Smolansky, A., Cram, A., Radulescu, C., Zeivots, S., Huber, E., & Kizilcec, R. F. (2023). Educator and student perspectives on the impact of Generative AI on assessments in higher education. In *Proceedings of the tenth ACM conference on Learning@ Scale* (pp. 378–382). <https://doi.org/10.1145/3573051.3596191>
- Solomon, C. J., & Papert, S. (1976). A case study of a young child doing turtle graphics in LOGO. In *Proceedings of the June 7–10, 1976, national computer conference and exposition* (pp. 1049–1056). <https://doi.org/10.1145/1499799.1499945>
- Solomon, C., Harvey, B., Kahn, K., Lieberman, H., Miller, M. L., Minsky, M., Papert, A., & Silverman, B. (2020). History of logo. *Proceedings of the ACM on Programming Languages*, 4(HOPL), 1–66. <https://doi.org/10.1145/3386329>
- Su, J., & Yang, W. (2023). Unlocking the power of ChatGPT: A framework for applying generative AI in education. *ECNU Review of Education*, 6(3), 355–366. <https://doi.org/10.1177/20965311231168423>
- Sun, J., Liao, Q. V., Muller, M., Agarwal, M., Houde, S., Talamadupula, K., & Weisz, J. D. (2022). Investigating explainability of Generative AI for code through scenario-based design. In *Proceedings of the 27th International Conference on Intelligent User Interfaces* (pp. 212–228). <https://doi.org/10.1145/3490099.3511119>
- Sunday, K., Oyelere, S. S., Agbo, F. J., Aliyu, M. B., Balogun, O. S., & Bouali, N. (2023). Usability evaluation of IMIKODE virtual reality game to facilitate learning of object-oriented programming. *Technology, Knowledge and Learning*, 28(4), 1871–1902. <https://doi.org/10.1007/s10758-022-09634-6>
- Tanty, H., Fernando, C., Valencia, J., & Justin, V. (2022). Critical thinking and problem solving among students. *Business Economic, Communication, and Social Sciences Journal (BECOSS)*, 4(3), 173–180. <https://doi.org/10.21512/becossjournal.v4i3.8633>
- Tran, N. D., May, J. J., Ho, N., & Ngo, L. B. (2023b). Exploring chatgpt's ability to solve programming problems with complex context. *Journal of Computing Sciences in Colleges*, 39(3), 195–209.
- Tran, A., Li, L., Rama, E., Angelikas, K., & MacNeil, S. (2023). Using large language models to automatically identify programming concepts in code snippets. In *Proceedings of the 2023 ACM Conference on International Computing Education Research* (pp. 22–23). <https://doi.org/10.1145/3568812.3603482>
- UNESCO. (2018). UNESCO ICT competency framework for teachers. *Report*, 2018. <https://unesdoc.unesco.org/ark:/48223/pf0000265721>
- Wermelinger, M. (2023). Using GitHub copilot to solve simple programming problems. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (pp. 172–178). <https://doi.org/10.1145/3545945.3569830>
- Woodley, X., Hernandez, C., Parra, J., & Negash, B. (2017). Celebrating difference: Best practices in culturally responsive teaching online. *TechTrends*, 61, 470–478. <https://doi.org/10.1007/s11528-017-0207-z>
- Wu, H., Fa, D., Wu, X., Tan, W., Chang, X., Gao, Y., & Weng, J. (2023). Research on the construction of intelligent programming platform based on AI-Generated content. In *Proceedings of the 15th International Conference on Education Technology and Computers* (pp. 9–15). <https://doi.org/10.1145/3629296.3629298>
- Wu, T. T., Silitonga, L. M., & Murti, A. T. (2024). Enhancing English writing and higher-order thinking skills through computational thinking. *Computers & Education*, 213, 105012. <https://doi.org/10.1016/j.compedu.2024.105012>
- Xiao, Y., & Watson, M. (2019). Guidance on conducting a systematic literature review. *Journal of Planning Education and Research*, 39(1), 93–112. <https://doi.org/10.1177/0739456X17723971>
- Xie, Y., Wu, S., & Chakravarty, S. (2023). AI meets AI: Artificial Intelligence and academic integrity—a survey on mitigating ai-assisted cheating in computing education. In *Proceedings of the 24th annual conference on information technology education* (pp. 79–83). <https://doi.org/10.1145/3585059>
- Yew, E. H., & Goh, K. (2016). Problem-based learning: An overview of its process and impact on learning. *Health Professions Education*, 2(2), 75–79. <https://doi.org/10.1016/j.hpe.2016.01.004>

- Yilmaz, R., & Yilmaz, F. G. K. (2023a). Augmented intelligence in programming learning: Examining student views on the use of ChatGPT for programming learning. *Computers in Human Behavior: Artificial Humans*, 1(2), 100005. <https://doi.org/10.1016/j.chbah.2023.100005>
- Yilmaz, R., & Yilmaz, F. G. K. (2023b). The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation. *Computers and Education: Artificial Intelligence*, 4, 100147. <https://doi.org/10.1016/j.caeai.2023.100147>
- Zacharis, N. Z. (2010). Measuring the effects of virtual pair programming in an introductory programming java course. *IEEE Transactions on Education*, 54(1), 168–170. <https://doi.org/10.1109/TE.2010.2048328>
- Zambrano, A. F., Liu, X., Barany, A., Baker, R. S., Kim, J., & Nasiar, N. (2023). From Ncoder to ChatGPT: From automated coding to refining human coding. In *International conference on quantitative ethnography* (pp. 470–485). Springer Nature Switzerland. <https://doi.org/10.1007/978-3-031-47014-1>
- Zhang, C., & Lu, Y. (2021). Study on artificial intelligence: The state of the art and future prospects. *Journal of Industrial Information Integration*, 23, 100224. <https://doi.org/10.1016/j.jii.2021.100224>
- Zhang, J., Li, D., Kolesar, J. C., Shi, H., & Piskac, R. (2022). Automated feedback generation for competition-level code. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (pp. 1–13). <https://doi.org/10.1145/3551349.3560425>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Jemimah Nathaniel^{1,2} · Solomon Sunday Oyelere³ · Jarkko Suhonen¹ · Matti Tedre¹

✉ Jemimah Nathaniel
jeminath@student.uef.fi; jemimah.nathaniel@covenantuniversity.edu.ng

Solomon Sunday Oyelere
s.oyelere@exeter.ac.uk

Jarkko Suhonen
jarkko.suhonen@uef.fi

Matti Tedre
matti.tedre@uef.fi

¹ School of Computing, University of Eastern Finland, N80101 Joensuu, Finland

² Department of Computer and Information Science, Covenant University, Ota, Ogun State, Nigeria

³ Department of Computer Science, University of Exeter, Exeter EX4 4RN, UK